

CS-4112 Introduction to Machine Learning

Evaluation Metrics

Arbish Akram

Department of Computer Science
Government College University

Is a Low Training Loss Enough?

- Training *minimises a loss* (SSE, BCE, MCE) with gradient descent.
- A low training loss does *not* mean the model is good. Recall the $M = 9$ polynomial: *zero* training error, very large test error.
- What we care about is *generalisation*: performance on *new, unseen* data.
- The right way to measure it depends on the *task* and on the *cost of each kind of mistake*.

Loss function: what the model *optimises* during training. It must be differentiable.

Evaluation metric: what *we* use to judge and compare trained models. It should match what the user cares about.

Training, Validation and Test Sets

Training set (70–80%)	Validation (10–15%)	Test (10–15%)
-----------------------	---------------------	---------------

- Common splits (train/validation/test): *80/10/10* or *70/15/15*.
- **Training set:** learns the weights $\mathbf{w}$.
- **Validation set:** chooses the *hyper-parameters*, the settings gradient descent does not learn: polynomial order M , regulariser λ , learning rate η , number of hidden neurons.
- **Test set:** used *once*, at the very end, to report the final performance.
- **Golden rule:** never use test data for training or for choosing hyper-parameters. Otherwise the reported score is over-optimistic. This mistake is called *data leakage*.

k -Fold Cross-Validation

Run 1	validate	train	train	train	train
Run 2	train	validate	train	train	train
Run 3	train	train	validate	train	train
Run 4	train	train	train	validate	train
Run 5	train	train	train	train	validate

 train  validate

- With little data, a large validation set wastes examples.
- Split the data into k equal *fold*s. Train k times; each run validates on a different fold and trains on the other $k - 1$.
- Final score = *average* of the k validation scores. Typical $k = 5$ or 10 .
- $k = N$ is *leave-one-out* cross-validation.
- Cost: training is repeated k times.

Measuring Regression Error

For N test examples with targets t_n and predictions y_n :

Metric	Formula
Mean Squared Error	$MSE = \frac{1}{N} \sum_{n=1}^N (y_n - t_n)^2$
Root MSE	$RMSE = \sqrt{MSE}$
Mean Absolute Error	$MAE = \frac{1}{N} \sum_{n=1}^N y_n - t_n $
Coeff. of determination	$R^2 = 1 - \frac{\sum_n (t_n - y_n)^2}{\sum_n (t_n - \bar{t})^2}$

- *RMSE* is in the same units as t . It is the E_{RMS} we used in curve fitting.
- *MAE* is less sensitive to a few very large errors (outliers).
- $R^2 = 1$: perfect. $R^2 = 0$: no better than always predicting the mean $\bar{t}$.

Accuracy, and Why It Is Not Enough

$$\text{Accuracy} = \frac{\text{number of correct predictions}}{\text{total number of predictions}}$$

$$\text{Error rate} = 1 - \text{Accuracy}$$

- Easy to understand, but it treats *every mistake as equally bad*. In practice, mistakes have very different costs:
 - Telling a cancer patient “you are healthy” is far worse than sending a healthy person for one more test.
 - Sending an important email to the spam folder is worse than letting one spam email through.
- To see *which* mistakes a model makes, we use the *confusion matrix*.

The Confusion Matrix

		Predicted	
		Positive	Negative
Actual	Positive	TP True Positive	FN False Negative
	Negative	FP False Positive	TN True Negative

 correct  mistake

- The *positive* class is usually the one we want to detect: disease, spam, fraud.
- FP is a *Type I error* (a false alarm).
- FN is a *Type II error* (a miss).

Positive/Negative is what the model *predicted*.
True/False tells whether that prediction was *correct*.

Example: A Spam Filter

A spam filter is tested on 100 emails: 40 are spam and 60 are not. It flags 35 emails as spam, of which 30 really are spam.

		Predicted	
		Spam	Not spam
Actual	Spam	TP = 30	FN = 10
	Not spam	FP = 5	TN = 55

- **TP:** spam correctly flagged = 30
- **FP:** good emails flagged = $35 - 30 = 5$
- **FN:** spam that was missed = $40 - 30 = 10$
- **TN:** good emails let through = $60 - 5 = 55$

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = \frac{30 + 55}{100} = 0.85$$

Precision and Recall

		Predicted	
		Spam	Not spam
Actual	Spam	TP = 30	FN = 10
	Not spam	FP = 5	TN = 55

— precision column - - - recall row

Precision: when the model says *positive*, how often is it right? It looks only at the *predicted*-positive column.

$$\text{Precision} = \frac{TP}{TP + FP} = \frac{30}{30 + 5} \approx 0.857$$

Recall (sensitivity, true positive rate): how many *actual* positives did the model find? It looks only at the actual-positive row.

$$\text{Recall} = \frac{TP}{TP + FN} = \frac{30}{30 + 10} = 0.75$$

High precision $\Rightarrow$ few *false alarms* (FP).

High recall $\Rightarrow$ few *misses* (FN).

Specificity and the False Positive Rate

		Predicted	
		Spam	Not spam
Actual	Spam	TP = 30	FN = 10
	Not spam	FP = 5	TN = 55

— actual-negative row

Specificity (true negative rate): how many *actual* negatives were correctly identified?

$$\text{Specificity} = \frac{TN}{TN + FP} = \frac{55}{60} \approx 0.917$$

False positive rate: the share of actual negatives that raised a false alarm.

$$\text{FPR} = \frac{FP}{FP + TN} = 1 - \text{Specificity} = \frac{5}{60} \approx 0.083$$

In medicine, a diagnostic test is reported by its *sensitivity* (= recall) and its *specificity*.

Which Mistake Is More Expensive?

Application	Prioritise	Why
Cancer screening	Recall	Missing a cancer (FN) can cost a life; a false alarm only means one more test.
Fraud detection	Recall	A missed fraud (FN) loses money; a flagged transaction can be checked by a person.
Email spam filter	Precision	An important email in the spam folder (FP) may never be read.
Videos recommended to children	Precision	Showing even one unsuitable video (FP) is unacceptable.

Before choosing a metric, ask: *which costs more here, a false positive or a false negative?*

The F1 Score

- One number that balances precision P and recall R : their *harmonic mean*

$$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2PR}{P + R}$$

- Spam filter:

$$F_1 = \frac{2(0.857)(0.75)}{0.857 + 0.75} = 0.80$$

Why not the ordinary average?

P	R	$\frac{P+R}{2}$	F_1
0.857	0.75	0.80	0.80
1.00	0.01	0.505	0.02

A model that almost never finds the positives ($R = 0.01$) still gets an average of 0.5. The harmonic mean is *small if either* P or R is small, so F_1 exposes it.

The Accuracy Paradox

A rare disease affects 1% of people. A test set has 1000 patients: 10 sick and 990 healthy. A “lazy” model always predicts healthy.

		Predicted	
		Sick	Healthy
Actual	Sick	TP = 0	FN = 10
	Healthy	FP = 0	TN = 990

- $\text{Accuracy} = \frac{0 + 990}{1000} = 99\%$
- $\text{Recall} = \frac{0}{0 + 10} = 0$
- It never finds a single sick patient.

Summary

- A low training loss is not enough: judge a model by its *generalisation* on unseen data.
- Split the data into training, validation and test sets (*80/10/10* or *70/15/15*). Use the test set only *once*; with little data, use *k-fold cross-validation*.
- Regression: *MSE*, *RMSE*, *MAE* and *R²*.
- Classification: the *confusion matrix* (TP, FP, FN, TN) shows which mistakes a model makes.
 - Precision = $\frac{TP}{TP + FP}$ Recall = $\frac{TP}{TP + FN}$ Specificity = $\frac{TN}{TN + FP}$
 - $F_1 = \frac{2PR}{P + R}$ is small if either *P* or *R* is small.
- *Accuracy* can be misleading on imbalanced data (the accuracy paradox).
- Choose the metric by asking: which costs more here, a *false positive* or a *false negative*?

Exercise

A COVID-19 test is evaluated on 500 people: 50 are infected and 450 are not. The test reports “infected” for 60 people, of whom 45 are actually infected.

1. Draw the confusion matrix.
2. Compute the accuracy, precision, recall, specificity and F1 score.